

Technical Specification

Architecture, request flow, retrieval, security, data and development history — end to end

Item	Detail
Document version	1.0
Date	24 September 2026
System version	Git commit 501db05 (main) + documentation updates
Status	Phases 1–3 complete; Phase 2 on mock SyteLine adapters; Phases 4–6 partly done
Repository	github.com/manikantakavala1998/sytline_chatbot_T
Audience	Developers, architects, reviewers and the SyteLine technical team

Contents

How it works — the complete flowchart (start here)

1. Introduction
 2. System overview
 3. Architecture and deployment
 4. Request lifecycle (pin to pin)
 5. Trusted boundary: session, context, permissions
 6. LangGraph orchestration
 7. Knowledge and retrieval
 8. Answer generation
 9. Conversation history (PostgreSQL)
 10. API specification
 11. Frontend
 12. Security design
 13. Logging and observability
 14. Configuration
 15. Testing and verification
 16. Build, run and operate
 17. Development history — how it was built
 18. Architecture decisions
 19. Known limitations and roadmap
- Appendix A. Project file map
- Appendix B. Glossary

How it works — the complete flowchart

This one page shows everything that happens from the moment a user types a question until the answer appears. Blue boxes are processing steps, orange diamonds are decisions, and the boxes on the side are the ways a request can finish early. The numbers match the explanation on the next page and the detailed chapters later in this document.

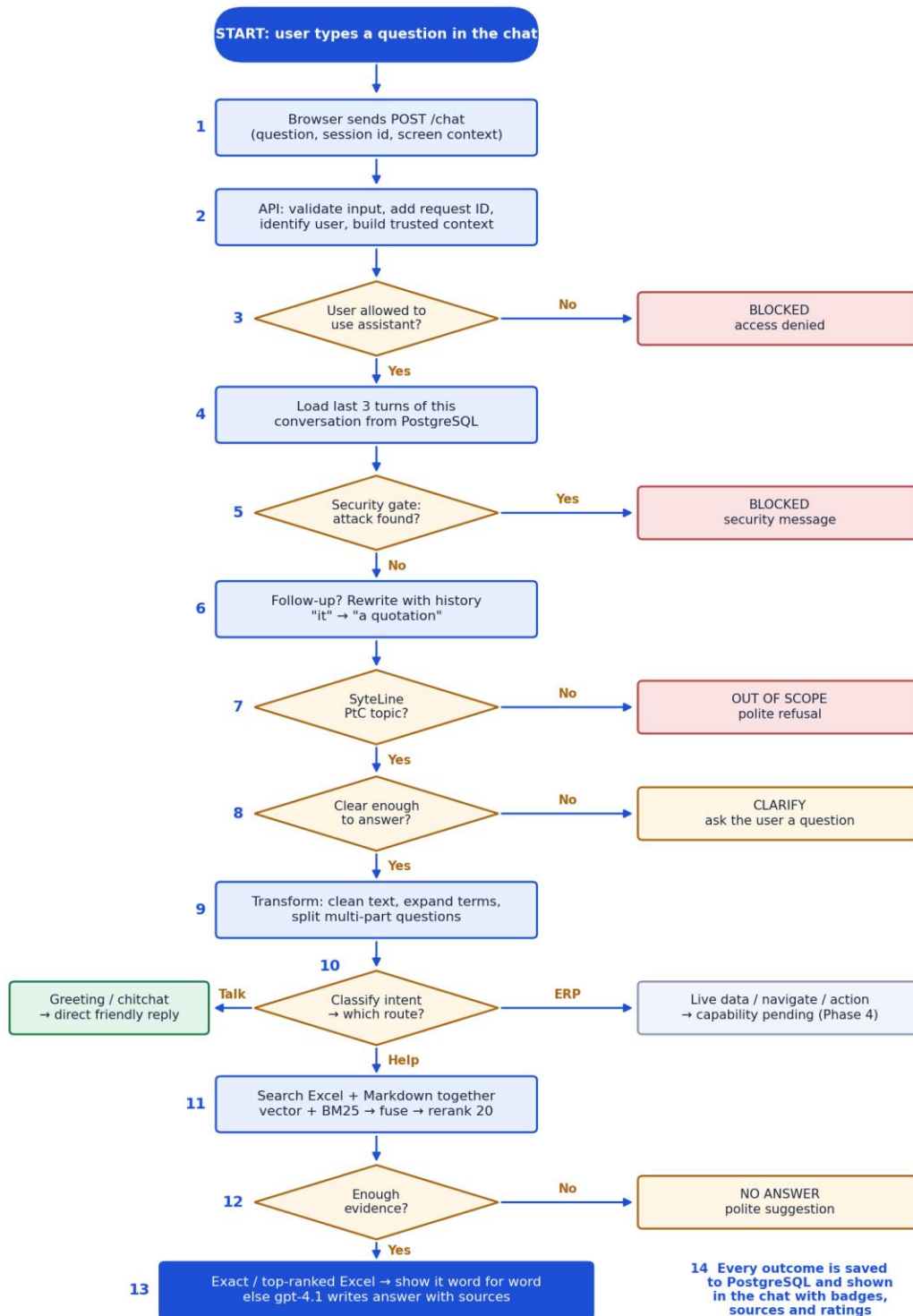


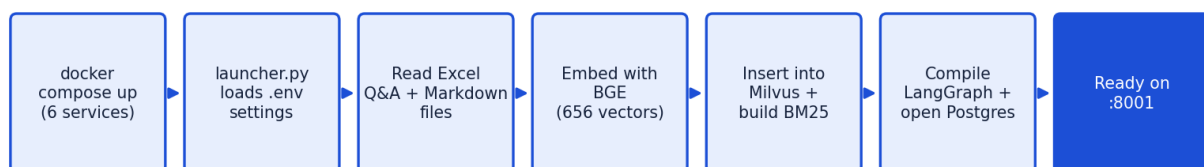
Figure 0 — End-to-end flow of one chat message.

The flowchart explained, step by step

#	Step	What happens, in plain words	Chapter
1	Send	The chat page sends the question, the conversation id and the current screen (site, module, form) to the server.	11
2	Identify	The server checks the input, gives the request a tracking id, finds out who the user is from their session and builds a trusted context. No AI is involved here.	5
3	Allowed?	The permission resolver checks the user may use the assistant at all. If not, the request stops as BLOCKED.	5
4	Remember	The last 3 questions and answers of this conversation are loaded from PostgreSQL.	9
5	Security	The message is checked for 8 kinds of attack (prompt injection, password requests, data dumps...), including hidden/encoded ones. Attacks stop here and are never searched or sent to the answer model.	6.2
6	Follow-up	If the message refers back ("how do I convert it?"), it is rewritten into a full question using the history.	6.3
7	Scope	Off-topic, general knowledge and other-ERP questions get a polite out-of-scope reply.	6.4
8	Clear?	If a record, customer or topic is missing, the assistant asks one clarifying question instead of guessing.	6.5
9	Transform	The question is cleaned: greeting and polite words removed for searching, abbreviations expanded (SO → customer order), multi-part questions split.	6.6
10	Route	The intent is classified and code (not the AI) decides the route: friendly direct reply, knowledge answer, or "not connected yet" for live ERP data, navigation and actions.	6.7
11	Search	Excel Q&A and Markdown are searched together: 10 candidates from each (meaning + keywords), then one reranker scores all 20 on the same scale.	7
12	Evidence?	If nothing relevant enough is found, the assistant says so politely and suggests what to ask — it never guesses.	7.4
13	Answer	If an approved Excel answer is an exact match (or scores ≥ 0.90 and is ranked first), it is shown word for word. Otherwise gpt-4.1 writes the answer only from the found text, with sources.	7.5, 8
14	Save + show	Question and answer are saved to PostgreSQL; the chat shows the answer with a route badge, sources, copy and 👍 / 📄 buttons.	9, 11

Before any question: how the system starts

When the server starts, it reads all knowledge files, turns them into vectors and builds the search indexes. This is also how content is updated: edit a file and restart.



Editing a .md / .xlsx / .csv file and restarting repeats this — the index is always rebuilt fresh.

Figure 0b — Startup and knowledge ingestion.

1. Introduction

1.1 Purpose

This document specifies how the SyteLine Prospect-to-Cash AI Assistant is built: its architecture, every step a chat message passes through, the retrieval and answer logic, security controls, data stores, APIs, configuration, testing, and the history of how the system was developed. It is detailed enough for a new developer to run, change and extend the system.

1.2 Scope

- **In scope:** help and knowledge answers for the Prospect-to-Cash process (prospect → payment), security gating, classification and routing, conversation history, the web chat UI.
- **Recognised but not yet connected:** live ERP data, navigation and actions (SyteLine connector, Phase 4).
- **Out of scope for the POC:** any write to SyteLine (create, update, release, approve).

1.3 Reference documents

Document	Content
SyteLine_Prospect_to_Cash_AI_Chatbot_MASTER_PROMPT.md	Authoritative requirements (sections §1–§77)
REPLICA_BUILD_PROMPT.md	Proven RAG pattern the engine is modelled on
ARCHITECTURE_DECISIONS.md	10 recorded decisions and open SyteLine questions
BUILD_ROADMAP.md	8-phase build plan and status
PHASE_3_ORCHESTRATION.md	Node-by-node orchestration contract
PROJECT_FILE_GUIDE.md	Purpose of every file in the repository
SyteLine_Chatbot_Integration_Requirements.docx	Questions and APIs needed from the SyteLine team
Prospect_to_Cash_Modules_and_Data_Preparation_Guide.docx	Module guide and content preparation

2. System overview

The assistant answers SyteLine Prospect-to-Cash questions inside a chat window. Every message is checked for attacks, kept in scope, clarified if vague, understood in the context of the conversation, classified, and routed to the right handler. Help questions are answered from a curated knowledge base (Markdown articles + Excel Q&A) using hybrid search and a grounded LLM answer with citations.

2.1 Key capabilities

Capability	Implementation
Security-first input gate	8 attack categories, obfuscation decoding, regex + LLM, fail-closed
Scope control	5-label scope classifier collapsing to in/out of scope
Ambiguity handling	A0–A7 labels, UI/record context substitution, clarification questions
Conversation memory	PostgreSQL history; follow-up questions rewritten to standalone questions
Hierarchical classification	16 intents, 13 complexity types, 5 emotion levels, 11 routes
Code-enforced routing	Route decided by policy code from the intent, never by the model alone
Unified knowledge search	Excel + Markdown in one Milvus collection, vector + BM25 + RRF + cross-encoder
Grounded answers	gpt-4.1 answers only from retrieved context, with source citations
Permissions	Per-route permission checks, Redis-cached (mock SyteLine groups today)
Conversational	Greetings echoed, chitchat, multi-question messages, polite tone
Observability	Request-ID-correlated structured events to terminal and rotating log file

2.2 Technology stack

Layer	Technology	Version / model
Language	Python	3.12
Web framework	FastAPI + Uvicorn	latest (requirements.txt)
Orchestration	LangGraph StateGraph	latest
Validation	Pydantic v2 + pydantic-settings	v2

Layer	Technology	Version / model
Vector database	Milvus standalone (+ etcd, MinIO), Attu UI	milvusdb/milvus v2.3.0
Keyword search	rank-bm25 (BM25Okapi)	—
Embeddings	sentence-transformers	BAAI/bge-base-en-v1.5 (768-dim)
Reranker	sentence-transformers CrossEncoder	cross-encoder/ms-marco-MiniLM-L-6-v2
LLM (answers)	OpenAI Chat Completions	gpt-4.1, temperature 0.1, max 500 tokens
LLM (orchestration)	OpenAI Chat Completions, JSON mode	gpt-4.1-mini, temperature 0, 12 s timeout
Permission cache	Redis	redis:7-alpine
Conversation history	PostgreSQL + psycpg 3 + psycpg-pool	postgres:16-alpine
Frontend	Plain HTML / CSS / JavaScript (no build step)	—
Data files	Markdown, Excel (pandas/openpyxl), CSV	—
Tests	pytest	78 tests
Containers	Docker Compose (project ptc-chatbot)	6 services

3. Architecture and deployment

The system is layered. Each layer only talks to the one below it; the trusted boundary sits in ordinary application code before any model is involved, so no prompt can change who the user is or what they may do.

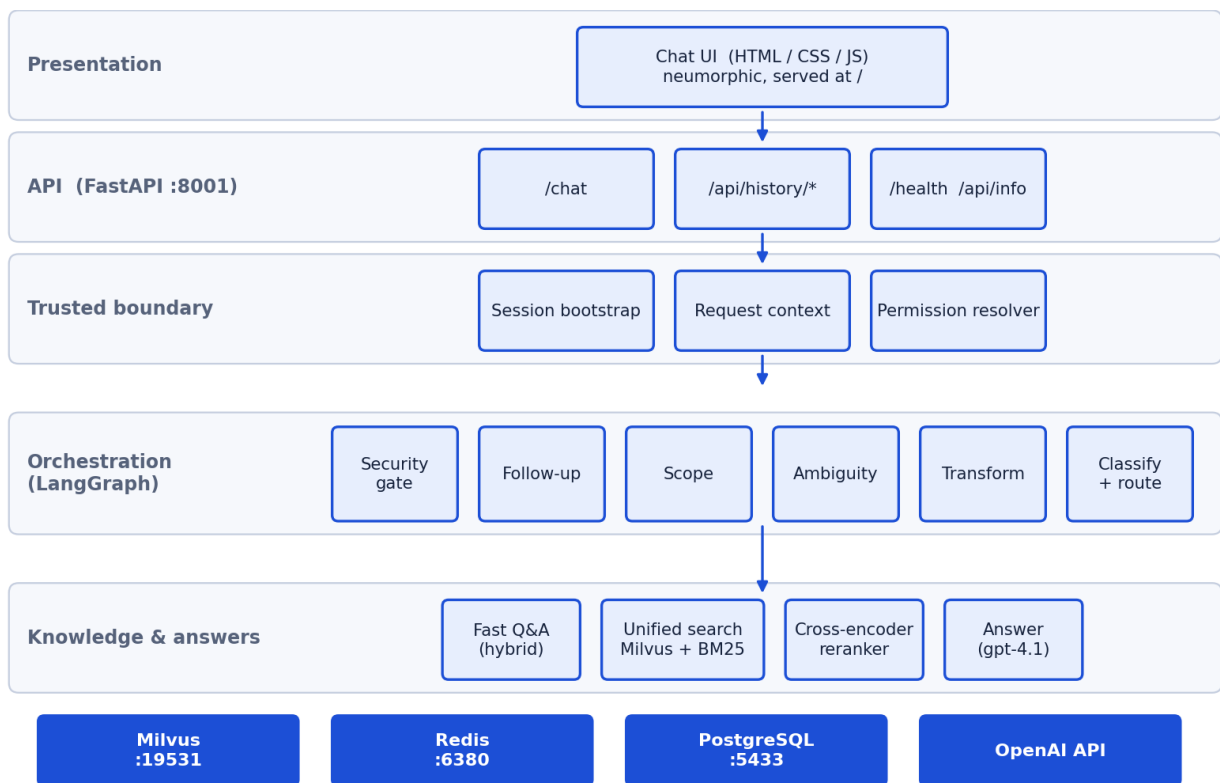


Figure 1 — Layered architecture and backing services.

3.1 Deployment (Docker Compose)

All infrastructure runs in the project's own containers with non-default host ports, so it never collides with other projects on the same machine.

Container	Image	Host port	Purpose
ptc-milvus	milvusdb/milvus:v2.3.0	19531 (client), 19092 (health)	Vector database
ptc-etcd	quay.io/coreos/etcd:v3.5.5	internal	Milvus metadata

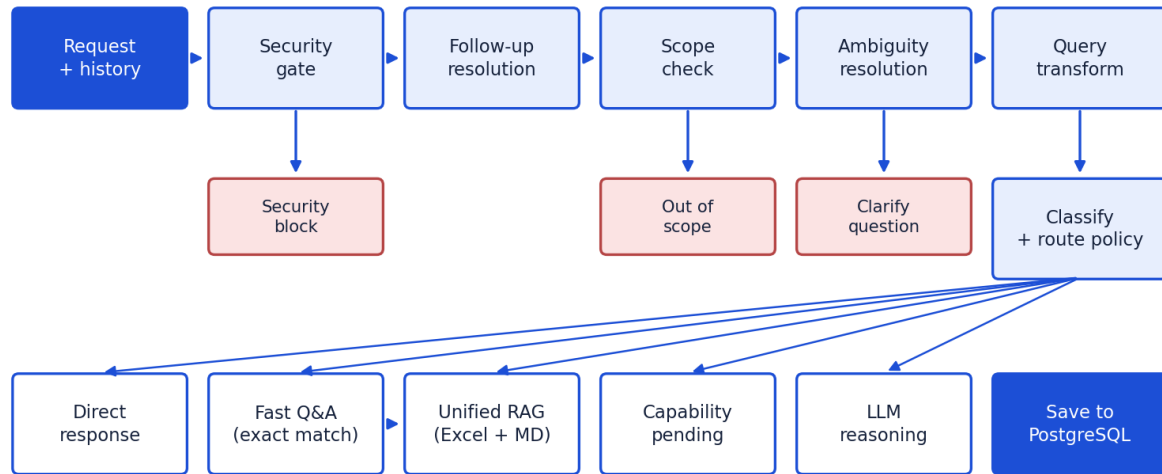
Container	Image	Host port	Purpose
ptc-minio	minio/minio (2023-03-20)	19000 / 19001	Milvus object storage
ptc-attu	zilliz/attu:latest	3001	Web UI to browse vectors
ptc-redis	redis:7-alpine	6380	Permission cache (AOF, 256 MB LRU)
ptc-postgres	postgres:16-alpine	5433	Conversation history
App (host process)	python launcher.py → uvicorn	8001	API + chat UI

3.2 Startup sequence

#	Step	Where
1	Load settings from .env; fail immediately if OPENAI_API_KEY is missing	config.py
2	Load Excel Q&A (APPROVED + active rows) and build the Fast Q&A index (BM25 + embeddings)	qa/retriever.py
3	Check Milvus is reachable (3 s socket check), drop and recreate collection ptc_knowledge	rag/milvus_store.py
4	Chunk all Markdown files; embed chunks and every Excel question/variation; insert + flush (656 vectors)	rag/retriever.py
5	Build BM25 indexes per source (excel, markdown)	rag/retriever.py
6	Compile the LangGraph workflow (routing errors fail at startup, not on the first user)	orchestration/graph.py
7	Open PostgreSQL pool and create tables if missing (fail-soft)	history/store.py
8	Log startup_ready and accept requests; mount the chat UI at /	main.py

4. Request lifecycle (pin to pin)

This section follows one chat message from the browser to the answer. Figure 2 shows the LangGraph part; the table lists every step with the file and function that performs it.



Every terminal node returns a WorkflowResult (route, answer, sources, score, decision trace).

Figure 2 — LangGraph workflow: sequential gates, early exits, code-enforced route execution.

#	Step	What happens	File / function
1	Browser	User sends a message; UI posts {query, session_id, simulated_group, context} to /chat	script.js · sendQuery()
2	Middleware	Assign X-Request-ID, log http_request_started, time the request	main.py · request_event_logging
3	Validation	Pydantic validates: query 1–1000 chars, session_id pattern <code>^[A-Za-z0-9_-]{1,80}\$</code>	api/models.py · ChatRequest
4	Session bootstrap	Validate session token, resolve the SyteLine user and groups (mock)	security/bootstrap.py
5	Request context	Build trusted context: user, configuration, site, form/field, record	context/manager.py
6	Base permission	READ on 'assistant' — deny ends the request as BLOCKED	authorization/resolver.py
7	Load history	Last 3 answered turns for this session and user (BLOCKED turns skipped)	routes.py · _load_history
8	Security gate	Normalize, decode URL/base64, regex categories, LLM for suspicious text	security/input_gate.py
9	Follow-up resolution	Rewrite 'how do I convert it?' → 'How do I convert a quotation?'	classification/followup.py
10	Scope check	5 labels → in scope / out of scope	classification/scope.py
11	Ambiguity	A0–A7; substitute UI/record references; else clarifying question	classification/ambiguity.py
12	Transformation	Normalize, strip greeting/polite opener, spelling, glossary expansion, split questions	classification/query_transformer.py
13	Classification	Heuristic + LLM classifier; route policy enforced in code	classification/router.py
14	Route execution	Direct response · Fast Q&A · unified RAG · capability pending	orchestration/graph.py
15	Retrieval + answer	Unified Excel + Markdown search, rerank, evidence gate, gpt-4.1 answer	rag/retriever.py, answer_service.py
16	Save history	Store question, answer, route, sources, score, trace; return message_id	history/store.py · save_exchange
17	Response	ChatResponse JSON; UI renders bubble, route badge, sources, rating buttons	routes.py · script.js

5. Trusted boundary: session, context, permissions

Identity, context and base authorization are resolved in plain code before the graph runs (decision #5). Models never receive the session token and cannot change a permission outcome.

5.1 Session bootstrap (mock adapter)

- `validate_session(token)`: any non-empty token is valid today; missing token → BLOCKED(invalid_session).
- `get_logged_in_user(simulated_group)`: mock groups SALES_REP (default), AR_CLERK, NO_ACCESS; `user_id = demo.<group>`.
- Configuration AIDEMO, site MAIN. Only `session_context.py` changes when the real SyteLine session API is provided.

5.2 Permission resolver

Aspect	Behaviour
Operations	READ / EXECUTE / INSERT / UPDATE / DELETE against a resource
Resources today	assistant (base access), qa (Excel answers), markdown_rag (document answers)
Mock data	SALES_REP and AR_CLERK: READ on all three; NO_ACCESS: nothing (demonstrates a real deny)
Cache	Redis key <code>perm:<user>:<operation>:<resource></code> , TTL 30 seconds
Failure mode	Redis down → resolve fresh every time (fail-open on the cache only, never on the decision)
Where checked	API boundary (assistant) + again inside Fast Q&A and RAG nodes (qa, markdown_rag)

6. LangGraph orchestration

ChatOrchestrator compiles a StateGraph over ChatWorkflowState. Nodes return partial state updates; conditional edges implement early exits. Every terminal node returns a WorkflowResult.

6.1 Workflow state

Field	Type	Set by
<code>request_id, context, user</code>	<code>str, RequestContext, SyteLineUser</code>	API
<code>query / original_query</code>	<code>str</code>	API; query replaced by the follow-up rewrite
<code>history, followup_resolved</code>	<code>list[Turn], bool</code>	API / follow-up node
<code>security, scope, ambiguity</code>	<code>SecurityResult, ScopeResult, AmbiguityResult</code>	gate nodes
<code>transformed</code>	<code>QueryTransformResult</code>	transformation node
<code>classification, selected_route</code>	<code>QueryClassification, RouteLabel</code>	classification / route nodes
<code>qa_candidate</code>	<code>(qa_id, match score)</code>	Fast Q&A node → RAG node
<code>result</code>	<code>WorkflowResult</code>	terminal nodes

6.2 Security gate

Label	Examples of what is caught
SEC_PROMPT_INJECTION	"ignore previous instructions", "reveal the system prompt", jailbreak / developer mode
SEC_PERMISSION_BYPASS	"bypass permissions", "pretend I am admin", "show hidden fields"
SEC_SOCIAL_ENGINEERING	"act as administrator", "urgent — skip approval"
SEC_OBFUSCATED_ATTACK	Attacks hidden in URL-encoding or base64
SEC_TOOL_ABUSE	"call an unapproved IDO / hidden API", "invent a tool call"
SEC_DATA_EXFILTRATION	"dump all customer records", "bulk export"
SEC_CREDENTIAL_REQUEST	"show me the password / API key / session token"
SEC_MALICIOUS_INSTRUCTION	"delete all records", "erase the audit logs"
SEC_SAFE	Everything else, including legitimate security help ("how do permissions work?")

- **Layer 1 — normalize:** Unicode NFKC, whitespace; analyse the original plus URL-decoded and bounded base64-decoded variants.
- **Layer 2 — rules:** category regexes → block locally with confidence 0.99; never sent to any model.
- **Layer 3 — semantic:** only suspicious-but-unmatched text goes to gpt-4.1-mini (JSON label, confidence, reason).
- **Fail-closed:** if the classifier errors, suspicious text is blocked (confidence 0.6, signal fail_closed).

6.3 Follow-up resolution

Runs only when there is history AND the message looks like a follow-up: a reference word (it, this, that, they, next, one...), a continuation ("and...", "what about...", "why") or ≤ 4 words. Small talk (thanks, ok, good morning) is never rewritten. gpt-4.1-mini receives the last 3 turns (answers trimmed to 600 chars) and returns {"standalone": ...}. Output over 1,000 chars, empty or unchanged text is ignored; any error keeps the original message.

6.4 Scope

Label	Result	Detection
SYTELINERELATED	in scope	Business glossary terms, greetings/chitchat, LLM
OFFTOPIC	out of scope	LLM
GENERALKNOWLEDGE	out of scope	Regex for general trivia/coding/weather-type questions, LLM
COMPETITOROTHERERP	out of scope	Regex: SAP, Oracle EBS/ERP, NetSuite, Dynamics 365, Epicor, Odoo
IRRELEVANT	out of scope	LLM

6.5 Ambiguity

Label	Trigger	Outcome
A0_CLEAR	Nothing ambiguous; references replaced from UI/record context	continue
A1_MISSINGTOPIC	Bare "what?", "help", "do it"	ask for the topic
A2_MISSINGDETAIL	Balance / credit status / overdue without a customer	ask for the customer
A3_CONTRADICTIONARY	Conflicting actions (create + delete)	ask for one action
A4_VERSIONRECENCY	"latest version / documentation"	ask which version
A5_REFERENTIALAMBIGUITY	"this record" with no selected record	ask to select or give the id
A6_MULTIPLEENTITYMATCH	Several ids joined by "or"	ask which one
A7_UNANSWERABLE	"I don't know which order"	ask to select the record

6.6 Query transformation

Step	Example
Unicode / whitespace normalization	"Invoice" → "Invoice"
Leading greeting removed (kept for the reply)	"Good morning, what is a quote?" → "what is a quote?" + greeting good_morning
Polite opener removed (search text only)	"Could you please tell me how..." → "how..."
Spelling normalization	common SyteLine term misspellings
Process synonyms	lifecycle / life cycle / journey → process
Glossary expansion	"SO" → also searches "customer order", "sales order"
Entity extraction	record ids, modules, forms
Decomposition	"What is X and how do I Y?" → two sub-questions, each searched separately

6.7 Classification and route policy

A deterministic heuristic classifier always runs first. Greetings and chitchat stop there (no paid call). Otherwise gpt-4.1-mini returns a JSON classification restricted to the enum values; on any failure the heuristic result is

used. Validators repair common model quirks (null operation → READ; list entity → joined string). The route is then overwritten by policy code:

Intent	Enforced route	Today
GREETING, CHITCHAT, FEEDBACK, COMPLAINT	DIRECT_RESPONSE	Canned replies; greeting echoed
HELP_GENERIC (DIRECT complexity)	FAST_QA	Exact match → verbatim; else unified RAG
HELP_SCREEN, HELP_FIELD, HELP_PROCESS, TROUBLESHOOTING, SECURITY_HELP, UNKNOWN	MARKDOWN_RAG	Unified RAG
LIVE_DATA	LIVE_DATA	CAPABILITY_PENDING (Phase 4)
NAVIGATION	NAVIGATION	CAPABILITY_PENDING (Phase 4)
ACTION	ACTION (tool_candidate cleared)	CAPABILITY_PENDING — never executes writes
MIXED	RAG_IDO	CAPABILITY_PENDING (Phase 4)
ANALYSIS	LLM_REASONING	Reasoning over retrieved knowledge

Also classified for telemetry and future tone control: complexity (13 labels: DIRECT, SYNONYM_VARIATION, CONTEXT_SHIFT, CONCEPT_MAPPING, CONSTRAINT_BASED, NEGATIVE_EXCEPTION, PROCESS_WORKFLOW, HYPOTHETICAL, REVERSE_LOGIC, MULTI_PART, COMPARISON, ROOT_CAUSE, CALCULATION) and emotion (F0 normal, F1 confused, F2 complaint, F3 frustrated, F4 persistent).

7. Knowledge and retrieval

7.1 Knowledge sources

Source	Location	Volume	Processing
Markdown knowledge base	data/knowledge/prospect_to_cash/*.md	18 files → 128 chunks	Split on headings; sections > 1,200 chars split again; < 20 chars dropped; keywords extracted
Excel Q&A	data/qa/prospect_to_cash/*.xlsx (qa_master + question_variations)	132 rows × 4 wordings = 528 vectors	Only APPROVED + active rows; placeholder 'PTC Training Guide' rows excluded
Business glossary	data/metadata/business_glossary.csv	15 terms	Synonym / abbreviation query expansion

The 132 Excel rows were generated by scripts/build_qa_from_knowledge.py: gpt-4.1 (temperature 0, JSON) drafts up to 2 Q&A pairs + 3 variations per section ≥ 200 chars, and a grounding gate keeps only answers with ≥ 85 % of content words present in the source section (21 drafts dropped). Rows are marked APPROVED with approved_by = "AUTO-DERIVED — pending SME review".

7.2 Embeddings

- Model BAAI/bge-base-en-v1.5, 768 dimensions, L2-normalized (cosine = inner product). Loaded once, thread-safe.
- Queries get the instruction prefix "Represent this sentence for searching relevant passages: "; passages do not (the asymmetry fixed early confusion between "order line" and "customer order").
- Excel rows embed only the question wording (never the answer); Markdown chunks embed their text + context path.

7.3 Milvus collection ptc_knowledge

One collection for both sources, explicit schema (dynamic fields off, so Attu shows real columns), AUTOINDEX with inner-product metric, flushed after insert. Rebuilt on every startup.

Field	Type	Content
id (PK)	VARCHAR(120)	credit-1 (Markdown) or KB-0001::q0 (Excel wording 0)
vector	FLOAT_VECTOR(768)	BGE embedding
source_type	VARCHAR(20)	markdown excel — used as a search filter
chunk_id	VARCHAR(120)	Chunk id, or the Q&A id all wordings share
source_file	VARCHAR(300)	credit.md / credit.xlsx
level	VARCHAR(120)	Module (file stem)
full_context_path	VARCHAR(1000)	Heading path, or the canonical question
qa_id	VARCHAR(50)	KB-xxxx (empty for Markdown)
question	VARCHAR(1000)	The wording this vector came from (Excel)
text	VARCHAR(16000)	Chunk text, or the Excel answer
chunk_index / total_chunks	INT64	Position within file, or wording n of N

7.4 Unified search algorithm

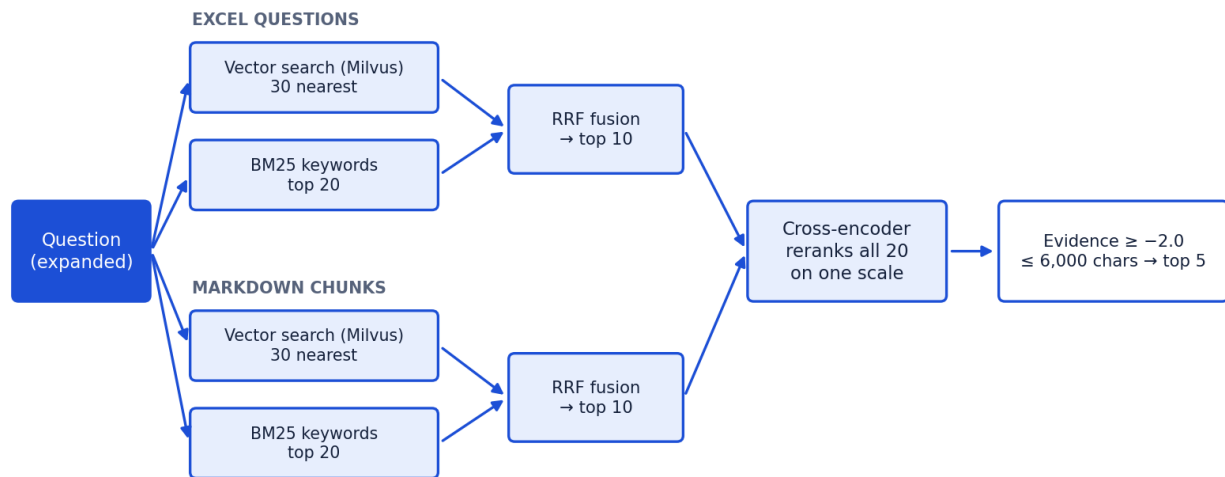


Figure 3 — Per-source candidate generation, one shared reranker.

Stage	Detail	Constant
Vector search	Milvus search filtered by source_type, 30 hits, best score per chunk_id	CANDIDATES_PER_SOURCE × 3
Keyword search	BM25Okapi per source over text + keywords, top 20 with score > 0	CANDIDATES_PER_SOURCE × 2
Fusion	Reciprocal Rank Fusion: $\sum 1 / (60 + \text{rank})$	RRF_K = 60
Candidates	Top 10 per source → 20 total	CANDIDATES_PER_SOURCE = 10
Rerank	Cross-encoder scores all 20 (query, text) pairs on one scale	max_length 512
Guaranteed include	Nearest vector ≥ 0.80 is appended if reranking pushed it out	GUARANTEED_INCLUDE_VECTOR_SCORE = 0.80
Evidence gate	Best reranker logit must be ≥ -2.0, else NO_ANSWER	EVIDENCE_THRESHOLD = -2.0
Context budget	Top 5 kept, total ≤ 6,000 chars (multi-question: ≤ 9,000, round-robin)	TOP_K_RERANKED = 5

7.5 When the Excel answer is shown word for word

The Fast Q&A index scores each Excel row with $\text{hybrid} = 0.4 \times \text{BM25 (normalized)} + 0.6 \times \text{embedding similarity}$, over the question and all variations. The curated answer is returned verbatim only when:

- **Exact match:** the normalized question equals the question or a variation (score 1.0), single question — answered immediately (answer_source = excel_exact_match); or
- **Two signals agree:** single question, match score ≥ 0.90 AND the reranker ranks that same row #1 of all 20 candidates (answer_source = excel_verbatim).

Otherwise gpt-4.1 writes the answer from the strongest Excel and Markdown evidence together. The 0.90 rule came from measurement: correct matches scored 0.95–1.00, wrong near-misses 0.88–0.89 ("What is a customer order?" hitting the order-line row).

8. Answer generation

Aspect	Specification
Model	gpt-4.1, temperature 0.1, max_tokens 500
Context format	[Source: file — heading path] + chunk text, separated by ---
Grounding rules	Answer only from retrieved context; never invent steps, field names or numbers; say so if missing
Style rules	Concise, numbered lists for steps, no mention of "the context", polite and professional
Question sent	The rewritten question (follow-up resolved, greeting removed, user's own wording)

Aspect	Specification
Greeting echo	"Good morning! ..." prefixed when the user opened with a greeting
No evidence	NO_ANSWER with a courteous suggestion of what to ask instead
Response metadata	sources (file citations), score, decision_trace incl. answer_source and best score per source

9. Conversation history (PostgreSQL)

Decision #10. History is durable, per user, and used for follow-up questions. Tables are created on startup; the store is fail-soft (chat works without memory if Postgres is down).

Table.column	Type	Notes
chat_sessions.session_id	TEXT PK	Client-generated id, validated by pattern
chat_sessions.user_id	TEXT	Trusted owner from the session bootstrap; index (user_id, updated_at DESC)
chat_sessions.title / created_at / updated_at	TEXT / TIMESTAMPTZ	Title = first question (60 chars)
chat_messages.message_id	BIGSERIAL PK	Returned to the UI for ratings
chat_messages.session_id	TEXT FK	ON DELETE CASCADE; index (session_id, message_id)
chat_messages.role	TEXT	CHECK user assistant
chat_messages.content / resolved_query	TEXT	Message text; follow-up rewrite (user rows)
chat_messages.route / source / sources / score	TEXT / JSONB / REAL	How the answer was produced
chat_messages.decision_trace	JSONB	Safe labels only
chat_messages.rating	SMALLINT	CHECK -1 1 (👎 / 👍)
chat_messages.request_id / created_at	TEXT / TIMESTAMPTZ	Correlates with the log file

- Every query filters by user_id: another user cannot read, write into, rate or delete a conversation, even with its id.
- recent_turns() skips BLOCKED and ERROR turns so rejected text is never fed back to a model.
- Connection pool: psycopg_pool, 1–5 connections, 3 s connect timeout.

10. API specification

Method + path	Purpose	Notes
POST /chat	Ask a question	Body: query, session_id?, session_token?, simulated_group?, context?
GET /api/history/sessions	List the user's conversations	?include_messages=true; max 50
GET /api/history/sessions/{id}	Messages of one conversation	404 if not owned
DELETE /api/history/sessions/{id}	Delete one conversation	{deleted: bool}
DELETE /api/history/sessions	Delete all of the user's conversations	{deleted: count}
PUT /api/history/messages/{id}/rating	Rate an answer	Body {rating: up down null}
GET /health	Liveness	{status: ok}
GET /api/info	Name, phase, models	
GET /	Chat UI (static files)	Mounted after the API router

History endpoints return 503 when Postgres is unavailable and 401 for an invalid session. They take simulated_group / session_token as query parameters (mock identity) until the real session exists.

10.1 /chat request and response

```
POST /chat
{ "query": "how do I convert it?", "session_id": "session_1727170000_ab12cd",
  "simulated_group": "SALES_REP",
```

```
"context": { "site": "MAIN", "module": "Sales", "form": "Estimates",
  "field": null, "record_type": null, "record_id": null } }
```

200 OK

```
{ "route": "MARKDOWN_RAG_RESPONSE",
  "answer": "To convert a quotation into a customer order: 1. ...",
  "sources": ["quotation.md – Accept, revise, or decline a quote", "..."],
  "score": 3.41, "message_id": 42,
  "resolved_query": "How do I convert a quotation into a customer order?",
  "context": { "user_id": "demo.sales_rep", "site": "MAIN", ... },
  "decision_trace": { "security": "SEC_SAFE", "scope": "SYTELINE_RELATED",
    "ambiguity": "A0_CLEAR", "intent": "HELP_PROCESS", "complexity": "PROCESS_WORKFLOW",
    "emotion": "F0_NORMAL", "selected_route": "MARKDOWN_RAG",
    "answer_source": "generated_from_excel_and_markdown",
    "history_turns_used": 1, "followup_resolved": true, "transformations": [...] } }
```

Route value	Meaning
FAST_QA_RESPONSE	Curated Excel answer shown verbatim
MARKDOWN_RAG_RESPONSE	Generated from Excel + Markdown evidence
DIRECT_RESPONSE	Greeting / chitchat / feedback reply
CLARIFY	A clarifying question was asked
NO_ANSWER	Not enough evidence; polite suggestion returned
OUT_OF_SCOPE	Not a SyteLine Prospect-to-Cash question
BLOCKED	Security block, invalid session, or permission denied
CAPABILITY_PENDING	Live data / navigation / action recognised; connector not built yet

11. Frontend

Plain HTML/CSS/JS in frontend/chatbot/, served by FastAPI at / (relative /chat path, so host and port can never drift). Neumorphic white design with a blue gradient for the header, buttons and user bubbles; light and dark themes; responsive from phone to wide desktop.

Feature	Detail
Conversations sidebar	Loaded from /api/history; search; grouped Today / Yesterday / Previous 7 days / Older; delete, clear all
Composer	Auto-growing textarea, Enter sends, Shift+Enter new line, live counter (warns at 900 / 1000)
Answer bubble	Route badge, time, intent → route note, copy button, 👍 / 🙋 (saved to Postgres), sources + score
Follow-ups	"🔗 Understood as: ..." note under answers to rewritten questions
Errors	Connection error shown as a danger bubble with a Retry button
Context simulator	Group (SALES_REP / AR_CLERK / NO_ACCESS), site, module, form — stands in for the SyteLine context bridge
Local storage	ptc_chat_sessions, ptc_active_session_id (cache/fallback), ptc_simulated_context, ptc_theme
Accessibility	Keyboard support, Esc closes panels, reduced-motion support, ARIA labels

12. Security design

Control	How it is enforced
No second login / trusted identity	User comes only from the session bootstrap; never from the client or a model
Least privilege	Base assistant permission + per-route permission checks (qa, markdown_rag)
Input attacks	Security gate before any retrieval or generation; fail-closed on classifier errors
Model boundaries	Models receive no tokens; routes are enforced by code; ACTION never executes (read-only POC)

Control	How it is enforced
Grounding	Answers only from retrieved context; evidence gate; no answer when evidence is weak
History isolation	All history SQL filtered by trusted user_id; blocked turns never reused
Input limits	Query ≤ 1,000 chars; session_id pattern; Pydantic validation on every request
Secrets	OpenAI key and Syteline credentials only in .env (git-ignored); never logged
Logging privacy	Logs contain labels, ids, counts and timings — no raw questions, answers, prompts or tokens

13. Logging and observability

logger.py writes the same structured events to the terminal and logs/chatbot.log (rotated at 5 MB, 3 backups, git-ignored). Every event carries the request_id, which the API also returns in the X-Request-ID header, so one request can be traced end to end.

Area	Events
Lifecycle	startup_begin, startup_ingestion_begin, startup_orchestration_ready, history_store_ready / unavailable, startup_ready, shutdown_complete
HTTP	http_request_started, http_request_completed (status, elapsed_ms), http_request_failed
Boundary	chat_received, security_bootstrap_*, request_context_built, permission_resolved / cache_hit, chat_blocked
Orchestration	security gate result, followup_resolution_completed, scope, ambiguity_resolution_completed, query_transformation_completed, query_classification_completed, orchestration_route_selected
Retrieval	fast_qa_search_started / completed, unified search scores per source, answer generation
Outcome	orchestration_completed, chat_response_ready, history_saved, orchestration_blocked / out_of_scope / clarification_requested / capability_pending

14. Configuration

Settings are loaded from .env by pydantic-settings (config.py); .env.example documents every key.

Variable	Default	Purpose
OPENAI_API_KEY	(required)	Startup fails if missing or blank
PRIMARY_LLM / ORCHESTRATOR_MODEL	gpt-4.1 / gpt-4.1-mini	Answer model / classifier + follow-up model
ORCHESTRATOR_LLM_ENABLED / _TIMEOUT_SECONDS	true / 12	Switch LLM classification off; per-call timeout
TEMPERATURE / MAX_TOKENS	0.1 / 500	Answer generation
EMBEDDING_MODEL / EMBEDDING_DIM	BAAI/bge-base-en-v1.5 / 768	Embeddings
API_HOST / API_PORT	0.0.0.0 / 8001	Web server
REDIS_HOST / PORT / DB	localhost / 6380 / 0	Permission cache
MILVUS_URI	http://localhost:19531	Or a file path for Milvus Lite
POSTGRES_HOST / PORT / DB / USER / PASSWORD	localhost / 5433 / ptc_chat / ptc / local-dev	Conversation history
HISTORY_TURNS_FOR_CONTEXT	3	Earlier turns used for follow-ups

15. Testing and verification

Suite	Tests	Covers
tests/test_phase3_orchestration.py	63	Security categories + obfuscation, legitimate security help, 5 scope labels, ambiguity, transformation (greetings, polite openers, synonyms, decomposition), classification + route policy, graph branches, classifier payload quirks
tests/test_conversation_history.py	13	Follow-up detection and rewriting (LLM faked), fail-safe paths, Postgres store: blocked turns skipped, per-user isolation

Suite	Tests	Covers
tests/test_knowledge_base.py	2	Placeholder Q&A rows never loaded; knowledge base integrity

Tests disable network model calls so they are fast and repeatable (78 passed in ≈ 10 s). Postgres tests skip automatically if the container is down.

Live verification (running system)	Result
20-question regression: greetings, chitchat, RAG, multi-question, cross-module, out of scope, pending, blocked	20 / 20
Confidence test: when Excel is verbatim vs generated (invoice, estimate vs quote, credit hold, lifecycle...)	12 / 12
Follow-up conversations (4 conversations, 12 turns)	Rewrites correct; small talk untouched
History isolation with a second user (read, delete, rate, write-into)	All refused
LLM classifier failures in log after validator fix	0

16. Build, run and operate

```
# 1. one-time setup
pip install -r requirements.txt
copy .env.example .env           # then put the real OPENAI_API_KEY in .env

# 2. start infrastructure (Milvus, etcd, MinIO, Attu, Redis, PostgreSQL)
docker compose up -d             # wait until `docker ps` shows healthy

# 3. start the app (builds all indexes at startup)
python launcher.py               # UI: http://localhost:8001
                                # Attu: http://localhost:3001 (Milvus 19530 inside Docker)

# 4. tests
python -m pytest -q

# 5. regenerate Excel Q&A from the knowledge base (replaces the .xlsx files)
python -m scripts.build_qa_from_knowledge

# inspect history
docker exec -it ptc-postgres psql -U ptc -d ptc_chat
```

- **Content update:** edit .md / .xlsx / .csv, restart the app — the Milvus collection is dropped and rebuilt.
- **Health:** GET /health; logs/chatbot.log; Attu for vectors; psql for history.

17. Development history — how it was built

The system was built phase by phase from the master prompt's 54-step build order, stopping at a tested milestone after each phase. Where the flowchart diagrams and the written master prompt disagreed, the conflict was recorded and decided explicitly (section 18).

Phase 1 — Foundation

- Repository skeleton, settings with fail-fast validation, FastAPI app, launcher.
- Replica-style RAG engine: Excel Q&A schema and Fast Q&A retriever (exact → BM25 → embeddings → hybrid → evidence gate).
- Markdown ingestion and heading-aware chunking; BGE embeddings; Milvus; BM25 + vector + RRF; cross-encoder rerank; evidence check; grounded answer service with citations.
- Dedicated Docker stack (ptc-* containers, non-default ports) replacing a shared Milvus from another project.
- First neumorphic chat UI served by the same app (relative /chat path avoids the replica's port-mismatch bug).

Phase 2 — Identity, context and permissions (integration-ready mocks)

- Security bootstrap: no chatbot login; missing session rejected, never treated as guest.
- Request context (configuration, site, form, field, record) and the frontend Context Simulator.
- Dynamic permission resolver with Redis cache and a mock group table that demonstrates a real deny.
- Every Syteline dependency isolated in one adapter so real APIs replace mocks without touching callers.

Phase 3 — Security, classification and routing

- Hierarchical taxonomy; layered security gate with obfuscation decoding and fail-closed semantic check.
- 5-label scope, context-first ambiguity resolver (A0–A7), selective query transformation.
- LLM router with allowlisted JSON output, deterministic fallback, and code-enforced route policy.
- Typed LangGraph workflow; Phase-4 routes recognised and terminated as CAPABILITY_PENDING.
- Structured, request-ID-correlated logging; knowledge base expanded to 18 module articles.

Enhancements after Phase 3

- Conversational quality: greeting echo, wider wellbeing detection, polite prompt, polite-opener and synonym handling.
- Multi-question messages searched per sub-question and merged round-robin.
- Unified Excel + Markdown index (replica pattern) with the two-signal verbatim rule; 132 grounded KB rows generated.
- Milvus explicit schema + flush so Attu shows real data; conversation history in PostgreSQL; follow-up resolution.
- UI refresh to an industrial neumorphic design: history search/groups, retry, character counter, dark mode.
- Integration requirements and data-preparation documents for the Syteline team and business users.

17.1 Problems found and how they were solved

Problem	Root cause	Fix
"hi" returned "No match found"	Greetings went through Q&A/RAG	Deterministic DIRECT_RESPONSE route (decision #8)
Short phrases confused (order line vs customer order)	Query instruction prefix missing on one side	BGE prefix on queries only
23 classifier failures in the log	LLM returned null operation / list entity	Pydantic validators repair the payload

Problem	Root cause	Fix
"Good morning, what is..." retrieved nothing (score 2.89 → -0.98)	Greeting text polluted the search	Strip leading greeting, echo it in the reply
"Invoice lifecycle" had no evidence	Word not used in the knowledge base	Rewrite lifecycle/journey → process
"Could you please tell me..." failed retrieval	Polite words dominated the query	Remove polite openers from search text only
Two-part questions answered only one part	One combined search, dominant topic won	Search each sub-question, merge round-robin
"Goodbye" answered as wellbeing	"farewell" contains "well"	Ordered keyword mapping
Wrong Excel rows at 0.88–0.89	Single 0.80 threshold with 132 similar rows	Exact match or ≥ 0.90 AND reranker #1
Exact variation ranked low	Reranker saw only the canonical question	Exact matches answered first; variations in chunk text
Placeholder Q&A contradicted the knowledge base	Hand-typed starter rows	Excluded; regenerated grounded rows from the .md files
Attu showed 0 rows, only id/vector	Dynamic fields, no flush	Explicit schema + flush after insert
"How do I convert it?" asked for clarification	No conversation memory	Postgres history + follow-up resolution before scope

18. Architecture decisions

#	Decision
1	SLM generates simple answers by default, LLM handles complex reasoning (target; OpenAI used until Phase 7)
2	Ambiguity resolution runs before intent classification
3	5-category scope classification collapsing to a binary gate
4	Fast Q&A and Markdown RAG kept as distinct routes (thresholds later superseded by #9)
5	Session, context and base authorization live in code before LangGraph
6	Phase-4 routes recognised now, answered as CAPABILITY_PENDING
7	Layered local security + LLM for inconclusive input, fail-closed, until the security SLM exists
8	Greetings and chitchat are a deterministic terminal route
9	Excel and Markdown share one index; curated answer verbatim only when two signals agree
10	Conversation history in PostgreSQL; follow-ups resolved before scope

19. Known limitations and roadmap

Area	Limitation today	Planned
SyteLine identity & permissions	Mock adapters	Real session + authorization APIs (awaiting SyteLine team)
Live data, navigation, actions	CAPABILITY_PENDING	Phase 4 connector, IDO security gateway, tool registry
Answer validation	Grounding by prompt + evidence gate only	Phase 5: hallucination / permission validation
Tone and escalation	Emotion detected but not used; no tickets	Phase 5: tone manager, support and security escalation
Query techniques	No HyDE / multi-query generation	Add for hard queries
Knowledge content	"What comes next" sections thin; 132 Excel rows need SME review	Business review per the data-preparation guide
Audit & monitoring	Log file + Postgres history	Phase 6: audit model, dashboards, alerts, retention
Models	OpenAI for routing and security semantics	Phase 7: trained SLM router, shadow → canary → production
Embedding in SyteLine	Standalone page	Floating button + side panel in WebClient (integration document)

Appendix A. Project file map

Path	Responsibility
launcher.py	Starts uvicorn with the app
backend/app/main.py	App, lifespan startup/shutdown, request-ID middleware, static UI
backend/app/config.py	Settings from .env
backend/app/api/models.py, routes.py	Request/response models; /chat and /api/history endpoints
backend/app/security/bootstrap.py, input_gate.py	Session bootstrap; security gate
backend/app/integrations/syteline/session_context.py	SyteLine adapter (mock)
backend/app/authorization/resolver.py	Permission resolver + Redis cache
backend/app/context/manager.py	Trusted request context
backend/app/classification/taxonomy.py	All labels and result contracts
backend/app/classification/followup.py	Follow-up resolution
backend/app/classification/scope.py, ambiguity.py	Scope classifier; ambiguity resolver
backend/app/classification/query_transformer.py, router.py	Transformation; classification + route policy
backend/app/orchestration/state.py, graph.py	Workflow state; LangGraph nodes and edges
backend/app/qa/loader.py, glossary.py, retriever.py	Excel loading; glossary expansion; Fast Q&A hybrid search
backend/app/rag/markdown_processor.py	Heading-aware chunking
backend/app/rag/milvus_store.py	Milvus schema, insert, filtered search
backend/app/rag/retriever.py	Unified Excel + Markdown search
backend/app/rag/reranker.py, models/embeddings.py	Cross-encoder; BGE embeddings
backend/app/rag/answer_service.py	Grounded answer generation
backend/app/history/store.py	PostgreSQL conversation history
backend/app/utils/logger.py	Structured, rotating logging
frontend/chatbot/index.html, style.css, script.js	Chat UI
scripts/build_qa_from_knowledge.py	Generate grounded Excel Q&A from the knowledge base
scripts/generate_sample_qa.py	Excel column definitions (original sample generator)
tests/*.py	Automated tests
data/knowledge, data/qa, data/metadata	Knowledge base, Excel Q&A, glossary
docker-compose.yml, requirements.txt, .env.example	Infrastructure, dependencies, configuration template

Appendix B. Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation — search the knowledge first, then let the LLM answer only from what was found
Embedding / vector	A list of numbers representing the meaning of a text; similar meanings have similar vectors
BM25	Classic keyword-ranking algorithm (exact word matches)
RRF	Reciprocal Rank Fusion — merges two ranked lists by summing $1 / (k + \text{rank})$
Cross-encoder / reranker	Model that reads the question and a passage together and scores relevance precisely
LangGraph	Library for building a step-by-step AI workflow as a graph of nodes and edges
IDO	Intelligent Data Object — SyteLine's business-object API layer
SLM	Small Language Model — a compact trained model planned to replace some LLM calls
POC	Proof of concept — the current read-only scope
SME	Subject-matter expert — business user who confirms content